FEN — Full Project Document & Operational System Guide

What FEN Is

FEN (Fast Earn Nearby) is a local-first micro-job marketplace designed around quick, simple, nearby tasks.

The entire platform is built around:

- simplicity
- speed
- low friction
- community help
- fast earning
- fast posting
- nearby availability

The purpose is to make local help feel immediate and approachable.

Instead of:

- hiring formal trades
- waiting days for quotes
- navigating complicated job platforms

FEN allows people nearby to quickly connect for:

- one-off jobs
- immediate help
- practical support
- short tasks
- local earning

Core Philosophy

The platform is intentionally:

- lightweight
- approachable
- not corporate
- not overcomplicated
- mobile-first
- friendly to non-technical users
- suitable for local communities

The app is meant to feel:

- immediate
- warm
- practical
- trustworthy
- calm
- premium without being intimidating

What FEN Is NOT

FEN is NOT:

- a trade directory
- a contractor marketplace
- an agency
- an employer
- a staffing platform
- a commercial advertising board
- a payment processor
- a guarantee of work quality

FEN simply connects local people.

The Main User Types

Posters

People who need help.

Examples:

- moving furniture
- collecting an item
- gardening help
- cleaning help
- dog walking
- companionship
- errands
- simple DIY help

Workers / Helpers

People looking for nearby earning opportunities.

Examples:

- students
- people wanting quick local money
- neighbours
- part-time earners
- casual helpers
- community members

Overall App Experience

The app should feel:

- smooth
- responsive
- premium
- clear
- uncluttered
- confidence inspiring

Every screen should:

- clearly explain what to do next
- avoid technical language
- reduce overwhelm
- keep actions obvious

Visual Design System

Primary Colours

Accent

Purple / violet:

```
#B56CFF
```

Backgrounds

```
#0B0714
#0E0A14
```

Typography Feel

The interface should feel:

- clean
- modern
- readable
- soft
- not aggressive
- not futuristic/neon

Interface Feel

The UI should use:

- rounded corners
- soft shadows
- layered cards
- subtle gradients
- clear spacing
- simple hierarchy

The app should avoid:

- clutter
- tiny buttons
- overloaded forms
- overly technical language

Application Flow

Startup Flow

```
Open app
↓
Splash screen
↓
Intro animation
↓
Auth session check
↓
Logged in?
 ├─ Yes → App tabs
 └─ No → Sign in
```

Intro System

Purpose

The intro establishes:

- identity
- trust
- polish
- consistency

Intro Asset

```
assets/images/fen_intro.mp4
```

Intro Animation Feel

The animation should feel:

- organic
- smooth
- premium
- calm
- slightly magical

The vine/pin concept represents:

- growth
- community
- local connection
- navigation
- nearby help

Authentication System

Sign In

Purpose

Allow existing users to quickly access the platform.

Layout

Elements

- logo
- app name
- subtitle
- email field
- password field
- sign in button
- create account button
- legal links

User Experience Goal

The sign in experience should:

- feel fast
- feel welcoming
- not overwhelm
- clearly explain errors

Sign In Process

```
Enter email/password
↓
Validate
↓
Supabase authentication
↓
Success → App
Failure → Friendly error
```

Sign Up

Purpose

Create a lightweight user profile quickly.

Required Information

Display Name

Used publicly inside the app.

Email

Used for login.

Password

Used for authentication.

Area/Postcode

Used for:

- nearby jobs
- travel estimates
- location awareness

Transport Mode

Used for:

- travel estimates
- future matching
- estimated arrival/travel

Transport Modes

```
Walk
Cycle
Drive
Public transport
Not sure yet
```

Platform Understanding Confirmation

The user must confirm they understand:

- FEN only connects people
- payments are direct
- users are responsible for safety

- FEN does not supervise jobs

Main App Navigation

The main navigation consists of:

```
Browse
Post
My Jobs
Messages
Profile
```

Each tab has a specific purpose.

Browse Tab

Purpose

The Browse tab allows users to:

- discover nearby work
- find quick earning opportunities
- see open jobs nearby
- explore categories
- estimate travel effort

Browse Layout

Header Area

Contains:

- title
- subtitle
- search

Example:

```
Browse nearby jobs
Find quick local tasks and earn nearby.
```

Search System

Users can search by:

- title
- category
- description
- area

The search should update live.

Browse Filters

Urgency Filters

```
Need now
Today
Flexible
All
```

These control urgency filtering.

Category Filters

```
Cleaning
Gardening
Moving
Delivery
Dog walking
Tech help
Admin help
Companionship
Other
```

The category chips should:

- visually highlight selection
- update immediately
- never feel broken

Distance Filtering

The distance filter exists to:

- help workers estimate practicality
- prioritise nearby work
- improve fast earning

Browse Map

Web Version

The web version uses:

- visible map preview placeholder
- nearby area information
- job summaries

The map placeholder exists because:

- full free web mapping is intentionally limited for MVP
- native experience is prioritised
- costs are kept at zero

Native Map

Native devices show:

- map pins
- nearby jobs
- approximate areas

The map must:

- never crash
- tolerate missing coordinates
- gracefully fallback

Browse Job Cards

Each card must clearly show:

- title

- category
- budget
- urgency
- area
- travel estimate
- open status

The card should immediately answer:

- what is the task?
- how much?
- where?
- how urgent?
- is it worth taking?

Browse Behaviour Rules

Only jobs with:

```
status = open
```

should appear as available.

Once a worker is selected:

- applications close
- Browse no longer treats the job as open

Post Tab

Purpose

The Post tab allows users to create jobs quickly.

The posting experience should feel:

- fast
- simple
- supportive
- intelligent
- low-pressure

Post Layout

Main Inputs

- title
- description
- category
- budget
- urgency
- optional time
- location
- optional photos placeholder
- safety notices
- post button

Smart Suggestions

The app should intelligently suggest:

- category
- cleaner descriptions
- realistic budgets

Examples:

```
walk dog → Dog walking
help with laptop → Tech help
move sofa upstairs → Moving / lifting
keep company → Companionship
```

Budget Logic

The app intentionally rounds pricing.

The goal is to feel:

- human
- simple
- realistic

Examples:

```
£10
£15
£20
£25
```

Avoid:

```
£13
£17
£19
```

Location Logic

Normal Jobs

Use:

```
Single area/postcode
```

Examples:

- cleaning
- gardening
- companionship
- assembly
- tech help

Transport Jobs

Require:

```
From postcode
To postcode
```

Examples:

- delivery
- collection
- pickup/dropoff

Important Location Rule

The system must understand:

```
Moving an item within the same home
≠ transport between locations
```

Photo System

Photos are currently:

- placeholder only
- not fully implemented

The app intentionally avoids:

- paid image infrastructure
- unnecessary storage costs

Moderation System

The moderation system exists to:

- reduce unsafe jobs
- stop business advertising
- avoid regulated work
- keep the platform local/simple

Jobs That Should Trigger Warnings

- gas work
- electrical work
- asbestos
- roofing
- dangerous work
- medical care
- childcare
- weapons
- drugs
- illegal jobs
- business advertising

Jobs That Should NOT Trigger Warnings

- companionship
- social support
- sitting with someone
- keeping company
- errands
- practical local help

Post Button Behaviour

The post button must:

- validate
- moderate
- create the job
- show loading
- show success/errors
- navigate correctly

The app must never silently fail.

Job Detail Screen

Purpose

The Job Detail screen is the main lifecycle hub for a job.

It controls:

- applications
- worker selection
- messaging
- cancellation
- progress
- status

Job Detail Layout

Main Information

- title
- category

- budget
- description
- area
- travel estimate
- status
- safety guidance

Application Lifecycle

Open

Workers can:

- apply
- message with application

Held

A worker has been selected.

The app should:

- close applications
- show selected worker
- stop new applications

Confirm Pending

The poster proposed a start time.

The worker must confirm.

In Progress

The job is actively happening.

Completed

The work is done.

Cancelled

The job was cancelled.

Chats become read-only.

Poster Controls

The poster can:

- edit
- cancel
- remove
- select worker
- propose start time

Worker Controls

The worker can:

- apply
- withdraw application
- confirm start time
- leave selected job before completion

Messaging System

Messaging exists to:

- coordinate
- agree details
- discuss timing
- confirm expectations

It should NOT replace:

- common sense
- safety judgement
- direct communication responsibility

Selected Worker Behaviour

Once selected:

- the job is held
- applications close
- other workers cannot apply
- Browse should stop showing it as open

Start Time System

The poster proposes.

The worker confirms.

The cancellation cutoff appears once timing is known.

Cancellation Cutoff

Purpose:

- protect workers from wasted travel
- reduce unfair cancellations

Calculated using:

- travel estimate
- transport mode
- 15 minute buffer

Remove Job Behaviour

Removing a job should:

- soft delete
- preserve data integrity
- not destroy active lifecycle state

My Jobs Tab

Purpose

My Jobs acts as:

- the user dashboard
- lifecycle management hub
- work history area

Sections

Posted by Me

Jobs created by the poster.

Applied For

Jobs the worker applied to.

Selected / Active

Jobs where the worker has been selected.

Completed / Cancelled

History section.

Clear Old Jobs

The platform intentionally allows:

- cleaning old history
- reducing clutter

But must never remove:

- active jobs
- held jobs
- ongoing work

Messages Tab

Purpose

The messaging system allows:

- agreement coordination
- timing discussion
- location clarification
- practical communication

Message List

Each conversation should clearly show:

- associated job
- status
- activity
- participant role

Conversation Behaviour

The conversation screen should:

- feel simple
- feel safe
- show status clearly
- allow practical coordination

Cancelled Conversations

When cancelled:

- messages become read-only
- warnings appear
- no further coordination should continue inside that job thread

---

Report System

The report system is intentionally lightweight for MVP.

Currently:

- local warning only
- no moderation queue yet
- users encouraged to keep screenshots

Profile Tab

Purpose

The profile system exists to:

- personalise matching
- improve trust
- support travel estimates
- improve community feel

Profile Data

Stored Information

- display name
- postcode
- transport mode
- bio
- completed jobs count

Transport Mode Importance

Transport mode affects:

- travel estimates
- practical nearby suitability
- future matching systems

Legal Pages

The legal pages are intentionally:

- simple
- readable
- non-corporate
- understandable

Legal Principles

Users must understand:

- FEN only connects people
- FEN is not the employer
- users arrange payments directly
- users are responsible for safety
- no guarantees exist

Messaging Architecture

Conversations are created:

- after worker selection
- tied to jobs
- tied to poster/worker relationship

The messaging system should:

- remain lightweight
- remain realtime where possible
- avoid unnecessary complexity

Database Structure

Jobs

Core job information:

- poster
- title
- category
- status

- budget
- location
- selected worker
- timing

Applications

Stores:

- worker interest
- application message
- application state

Conversations

Stores:

- poster-worker relationship
- job linkage
- archive state

Messages

Stores:

- realtime communication
- sender
- timestamps
- content

Travel Estimate System

Travel estimates are intentionally lightweight.

The system uses:

- postcode geocoding
- GPS when available
- transport mode

No paid mapping infrastructure is used.

Performance Goals

The app should:

- load quickly
- remain responsive
- feel lightweight
- work well on lower-end devices

Reliability Goals

Every clickable element must:

- visibly respond
- show loading
- show success/error
- never silently fail

Engineering Philosophy

The project intentionally avoids:

- unnecessary complexity
- expensive infrastructure
- overengineering
- enterprise-level systems too early

The MVP prioritises:

- working core flow
- stability
- simplicity
- real-world usability

Long-Term Vision

Future systems may include:

- push notifications
- better matching
- fit scores
- reviews
- verification
- image uploads

- in-app payments
- smarter routing
- recommendation systems
- moderation dashboard

But the MVP should remain:

- simple
- fast
- local
- low cost
- understandable

Final Core Principle

Everything inside FEN should support this core idea:

```
Someone nearby needs help.
Someone nearby wants to earn.
The platform should make that connection feel fast, simple, and natural.
```